

1 SoC-Grundlagen und Design Flow

Flow: System Planning & HDL Design → RTL Synthesis → Netlist Linking & Constraints → Packing / Placement / Optimization → Routing → Post-Route Optimization → Static Timing Analysis → Bitstream Generation

Prüfungsregel: Timing-Constraints wirken bereits ab der Synthese und werden in allen Folgeschritten berücksichtigt.

Die Zynq-7000 Plattform integriert Prozessor und FPGA auf einem Chip:

- **Processing System (PS):** ARM Cortex-A9, Speichercontroller, feste Peripherie
- **Programmable Logic (PL):** FPGA-Logik für applikationsspezifische Beschleuniger

PS und PL sind über **AXI-Interfaces** gekoppelt und erlauben Hardware/Software-Co-Design.

Die PL stellt folgende Kernressourcen bereit:

- **Logik:** CLBs mit LUTs und Flip-Flops
- **Speicher:** Block RAM (synchron)
- **Arithmetik:** DSP-Slices (MAC, Multiplizierer)

Diese Ressourcen ermöglichen hochparallele, datenpfadorientierte Implementierungen. main Challenges: HW/SW Partitioning, Timing Closure bei hoher Integration, Verifikation über Hardware und Software, Tool- und Design-Flow-Komplexität.

2 SoC Constraints

2.1 Rolle von Constraints im SoC-Design

HDL beschreibt primär Funktionalität, nicht physikalische oder zeitliche Eigenschaften der Hardware. Moderne SoC-Designs sind ohne explizite Constraints nicht korrekt implementierbar.

Merksatz (Prüfung): Constraints liefern dem Tool alle Informationen zu Pins, Spannungen und Zeiten, die im HDL fehlen.

Constraints steuern Synthese, Platzierung, Routing und Timing-Optimierung.

2.2 Arten von Constraints

Constraints lassen sich in drei Hauptkategorien einteilen:

- **Physikalische Constraints:** Pin-Zuordnung, I/O-Standards, Nutzung spezifischer FPGA-Ressourcen, Platzierung von Zellen
- **Timing Constraints:** Clock-Definition, Input- und Output-Delays, Timing-Ausnahmen
- **Configurations-Constraints:** Bitstream-Optionen, Konfigurationsmodi

Prüfungsregel: Timing - Constraints wirken bereits in der Synthese, physikalische Constraints primär in der Implementierung.

2.3 XDC-Format und Constraint-Organisation

Vivado verwendet XDC-Dateien auf Basis von Synopsys Design Constraints (SDC). Constraints folgen Tcl-Syntax und werden **sequentiell** interpretiert.

Empfohlene Struktur:

- Timing Assertions (Clocks, I/O-Delays)
- Timing Exceptions (False Paths, Multicycle Paths)
- Physikalische Constraints (Pins, I/O-Standards)

Merksatz: Timing zuerst, Pins zuletzt.

2.4 Physikalische Constraints

Physikalische Constraints definieren die Abbildung logischer Ports auf physische Pins sowie elektrische Eigenschaften:

- Pin-Zuweisung (PACKAGE_PIN)
- I/O-Standard (IOSTANDARD)
- Pull-Ups, Pull-Downs

Pad-Typen (Prüfung)

- Single-Ended Pads (z.B. LVCMOS)
- Differenzielle Pads (z.B. LVDS)

XDC-Code-Schablone

```
1 set_property IOSTANDARD LVCMOS33 [get_ports rst]
2 set_property PULLUP true [get_ports rst]
3 set_property PACKAGE_PIN V13 [get_ports rst]
4 ## Timing exceptions (Exam)
5 set_false_path -from [get_ports rst]
6 # Multi-cycle (example: 2 cycles for setup, 1 for hold)
7 set_multicycle_path 2 -setup -from [get_cells regA] -to [get_cells regB]
8 set_multicycle_path 1 -hold -from [get_cells regA] -to [get_cells regB]
```

In Sonderfällen können explizite Platzierungsconstraints (LOC, BEL) verwendet werden.

2.5 Timing-Grundlagen und Static Timing Analysis (STA)

Timing-Verifikation erfolgt mit **Static Timing Analysis (STA)** ohne Simulation.

Zentrale Begriffe:

- **Clock Latency:** Verzögerung von Clock-Quelle zu Register
- **Clock Skew:** Laufzeitunterschied zwischen Clock-Pfaden
- **Clock Jitter:** Zeitliche Schwankung der Clock-Flanke
- **Slack:** Zeitreserve eines Pfades

Timing-Bedingungen:

- **Setup:** Daten müssen vor aktiver Clock-Flanke stabil sein
- **Hold:** Daten müssen nach Clock-Flanke stabil bleiben
- **Pulse Width:** Mindestbreite der Clock

Prüfungsregel: Timing ist korrekt, wenn Setup- und Hold-Bedingungen erfüllt sind und der $Slack \geq 0$ ist.

STA analysiert:

- Clock-to-Clock-Pfade
- Minimale und maximale Pfade
- Alle gültigen Registerpfade

Wichtige Kennzahlen:

- **WNS (Worst Negative Slack)**
- **TNS (Total Negative Slack)**

Prüfungsmerksatz: STA betrachtet alle relevanten Pfade unabhängig von Stimuli.

Setup- und Hold-Bedingungen (Prüfung):

$$\text{Setup: } T_{clk} \geq T_{cq}^{max} + T_{comb}^{max} + T_{setup} - T_{skew}$$

$$\text{Hold: } T_{cq}^{min} + T_{comb}^{min} \geq T_{hold} + T_{skew}$$

Slack (beide Fälle):

$$\text{Slack} = \text{Required Time} - \text{Arrival Time}$$

Merksatz: Setup prüft max-Pfade, Hold prüft min-Pfade.

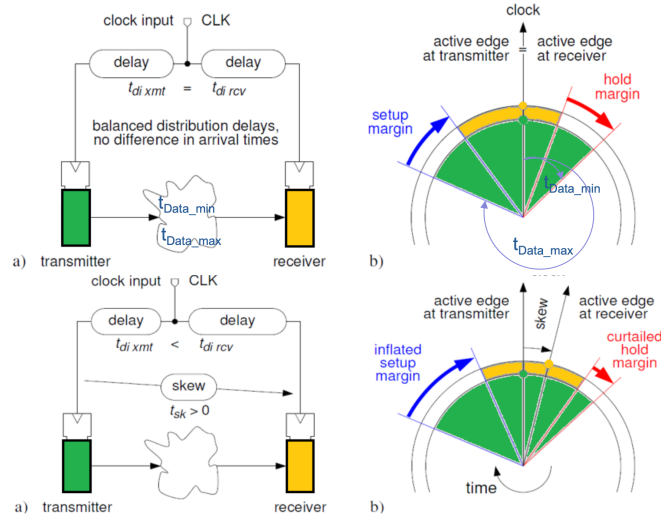


Abbildung 1: Timing-Nichtidealitäten: Latency, Skew und Jitter.

CDC – 2-FF Synchronizer (Exam)

```
1 process(clk_b)
2 begin
3   if rising_edge(clk_b) then
4     sync1 <= sig_a;
5     sync2 <= sync1;
6   end if;
7 end process;
```

Pipeline-Register (Timing-Fix)

```
1 process(clk)
2 begin
3   if rising_edge(clk) then
4     stage1 <= din;
5     stage2 <= stage1;
6   end if;
7 end process;
```

2.6 Clock-Definition

Jede relevante Clock muss explizit definiert werden.

Formel

$$T_{clk} = \frac{1}{f}$$

Code-Schablone

```
1 create_clock -name clk -period 10 [get_ports clk]
```

Duty-Cycle / Waveform (Prüfung):
Nicht-50% Duty-Cycle explizit angeben:

```
1 create_clock -period 5 -waveform {0 2} [get_ports clk]
```

Merksatz: Ohne -waveform wird 50% angenommen.

2.7 Input-Delay-Constraints

Externe Signalankunftszeiten müssen spezifiziert werden.

Prüfungsformel

$$t_{input_delay} = t_{data} - t_{clk}$$

Prüfungs-Schema (min/max):

- -max: Setup-Bedingung
- -min: Hold-Bedingung

Slack (Prüfung):

$$\text{slack} = \text{required time} - \text{arrival time}$$

Code-Schablone

```
1 set_input_delay -clock clk 3 [get_ports data_in]
```

2.8 Output-Delay-Constraints

Output-Delays definieren erforderliche externe Setup- und Hold-Zeiten.

Code-Schablone

```
2 set_output_delay -clock clk 5 [get_ports data_out]
```

2.9 Wirkung von Constraints

Constraints beeinflussen Platzierung, Routing und Optimierung direkt.

Abschluss-Merksatz (Prüfung): Ohne korrekte Constraints kein valides Timing und kein reproduzierbares Design.

3 System Level VHDL und IP-Wiederverwendung

3.1 Zielsetzung von System Level VHDL

Moderne SoC-Entwicklung erfordert skalierbare, parametrisierbare und wiederverwendbare Hardwarebeschreibungen. System Level VHDL adressiert Komplexität, Heterogenität und Time-to-Market durch Hierarchie, Modularisierung und Abstraktion.

Merksatz (Prüfung): System Level VHDL zielt auf Wiederverwendung und Skalierung, nicht auf Gate-Optimierung.

Ziel ist die Wiederverwendung bewährter IP-Blöcke statt monolithischer, hardcodierter Designs.

3.2 Grundprinzipien der Wiederverwendung

Wiederverwendbarer VHDL-Code folgt klaren Regeln:

- Keine fest kodierten Parameter
- Klare Schnittstellen
- Parametrisierung über Generics
- Zentrale Definition von Konstanten und Typen
- Saubere Dokumentation und Kommentare

Prüfungsregel: Projektweit konstant $\Rightarrow$ Package. Variabel pro Instanz $\Rightarrow$ Generic.

Code wird typischerweise von anderen Entwicklern genutzt als vom ursprünglichen Autor. Lesbarkeit ist ein funktionales Kriterium.

3.3 Alias, Konstanten und Generics

Aliases verbessern die Lesbarkeit, insbesondere bei Vektorslices.

Alias – Code-Schablone

```
1 alias opcode : std_logic_vector(3 downto 0) is instr(15 downto 12);
```

Konstanten definieren unveränderliche Entwurfsparameter und erhöhen Konsistenz.

Konstante – Code-Schablone

```
3 constant DATA_WIDTH : integer := 16;
```

Generics ermöglichen die Übergabe statischer Parameter von außen in eine Entity.

Generic – Code-Schablone

```
1 entity alu is
2   generic ( WIDTH : integer := 16 );
3   port (
4     a, b : in  std_logic_vector(WIDTH-1 downto 0);
5     y    : out std_logic_vector(WIDTH-1 downto 0)
6   );
7 end entity;
```

Merksatz: Generics werden bei der Instanziierung festgelegt und sind zur Laufzeit konstant.

3.4 Generate-Konstrukte

Generate-Anweisungen erlauben die strukturierte Erzeugung mehrfacher Instanzen. In Kombination mit Generics entstehen skalierbare Hardwarestrukturen.

Typische Einsatzfälle:

- Replikation identischer Verarbeitungseinheiten
- Parametrisierbare Breiten und Tiefen
- Strukturierte Arrays von Modulen

Prüfungsregel: Generate ist elaborationszeitlich wirksam und erzeugt echte Hardware.

Generate – Code-Schablone

```
4 gen_loop : for i in 0 to N-1 generate
5   u : entity work.reg
6   port map (clk => clk, d => din(i), q => dout(i));
7 end generate;
```

3.5 Funktionen in VHDL

- Nur Eingangsparameter
- Genau ein Rückgabewert
- Keine Signalzuweisungen, kein wait

Geeignet für kombinatorische Logik.

3.6 Prozeduren in VHDL

Mehrere Ein- und Ausgänge möglich. Primär für Testbenches, Synthese toolabhängig.

3.7 Arrays und Records

Records gruppieren Elemente unterschiedlicher Typen zu logischen Strukturen. Sie eignen sich für Busse, Transaktionen und strukturierte Datensätze im Systementwurf.

Record – Code-Schablone

```
1 type bus_t is record
2   addr : std_logic_vector(15 downto 0);
3   data : std_logic_vector(31 downto 0);
4   we   : std_logic;
5 end record;
```

3.8 Packages

Zentrale Container für Konstanten, Typen und Funktionen. Fördern Wiederverwendung und Konsistenz.

Package – Minimal-Schablone

```
8 package defs is
9   constant WIDTH : integer := 16;
10  function inc(x : integer) return integer;
11 end package;
12
13 package body defs is
14  function inc(x : integer) return integer is
15  begin
16    return x + 1;
17  end function;
18 end package body;
```

Package verwenden

```
1 use work.defs.all;
```

Typische Anwendung: Projekt-Settings, mathematische Hilfsfunktionen, Protokollparameter.

3.9 System-Level-Designwirkung

System Level VHDL verschiebt den Fokus von signalorientierter Beschreibung hin zu strukturierter Systemarchitektur. Es bildet die Grundlage für IP-basierte SoC-Designs und saubere HW/SW-Partitionierung.

Abschluss-Merksatz: Gute Wiederverwendung entsteht durch klare Schnittstellen, Generics und Packages.

4 Fixed Point Arithmetic

4.1 Motivation und Problemstellung

Reale Zahlen können in Hardware nicht direkt verarbeitet werden. Fixed-Point-Arithmetik approximiert reelle Werte mit endlicher Wortlänge.

Merksatz (Prüfung): Bei Fixed Point ist der Entwickler verantwortlich für Skalierung, Wortlänge und Überlauf.

4.2 Fixed-Point Formate und Bitwachstum (Prüfungsformeln)

Qn.m-Notation

- n : Integer-Bits (inkl. Vorzeichen bei signed)
- m : Fractional-Bits

Auflösung

$$\Delta = 2^{-m}$$

Wertebereich

$$uQ_{n,m} = [0, 2^n - 2^{-m}] \quad sQ_{n,m} = [-2^{n-1}, 2^{n-1} - 2^{-m}]$$

4.3 Ganzzahl- und Zweierkomplement-Darstellung

Für n Bit:

- Unsigned: $[0, 2^n - 1]$
- Signed: $[-2^{n-1}, 2^{n-1} - 1]$

Prüfungsregel: Bitmuster identisch, nur der Typ bestimmt die Interpretation.

4.4 Fixed-Point-Zahlendarstellung (Qn.m)

Fixed-Point-Zahlen besitzen einen impliziten Dezimalpunkt.

- **uQn.m**: unsigned
- **sQn.m**: signed

4.5 Skalierung und Typkonvertierung

Prüfungsformeln

$$\text{Fixed} = \lfloor \text{Real} \cdot 2^m \rfloor \quad \text{Real} = \text{Fixed} \cdot 2^{-m}$$

Prüfungsregel: Ohne explizite Typkonvertierung keine Arithmetik in VHDL.

4.6 fixed_pkg vs. klassische Umsetzung

fixed_pkg bietet:

- Automatisches Alignment
- Guard Bits
- Rundung und Sättigung

Prüfungsfokus: Prüfungen verlangen meist klassische Umsetzung ohne fixed_pkg.

4.7 Addition und Subtraktion

Formatregel (Prüfung)

$$Q(n_1.m_1) + Q(n_2.m_2) = Q(\max(n_1, n_2) + 1). \max(m_1, m_2)$$

Vorgehen ohne fixed_pkg

1. Fractional Bits angleichen
2. Vorzeichenerweitern
3. Addieren
4. Optional: Trunkieren/Runden

Additions-Code-Schablone

```
1 A_ext <= signed(A) sll 1; -- Guard Bit
2 B_ext <= signed(B);
3 Y_ext <= A_ext + B_ext;
```

Prüfungsregel: Fehlende Guard Bits = häufigste Fehlerquelle.

4.8 Multiplikation

Formatregel

$$Q(n_1.m_1) \cdot Q(n_2.m_2) = Q(n_1 + n_2).(m_1 + m_2)$$

Signed $\times$ signed:

$$Q(n_1 + n_2 - 1).(m_1 + m_2)$$

Multiplikations-Code-Schablone

```
1 P_ext <= signed(A) * signed(B);
```

Prüfungsregel: Nach der Multiplikation sind meist zu viele Bits vorhanden.

Typischer Fixed-Point-Workflow

```
1 Scale -> Sign-Extend + Guard -> Compute -> Truncate/Round -> Slice
```

4.9 Überlauf, Rundung und Sättigung

- Wrap-Around
- Sättigung
- Trunkierung
- Rundung

Prüfungsfokus: Trunkierung vs. Rundung sicher unterscheiden.

4.10 Rechenbeispiel: Bitmuster, Fixed Point und Skalierung

Gegeben:

```
1 value = "11100111"
```

a) unsigned(7 downto 0)

$$11100111_2 = 2^7 + 2^6 + 2^5 + 2^2 + 2^1 + 2^0 = 231$$

b) signed(7 downto 0)

$$231 - 2^8 = -25$$

c) ufixed(2 downto -5)

$$\Delta = 2^{-5}, \quad x = 231 \cdot 2^{-5} = \frac{231}{32} = 7.21875$$

d) sfixed(2 downto -5)

$$x = (-25) \cdot 2^{-5} = \frac{-25}{32} = -0.78125$$

4.11 Rechenbeispiel: Real $\rightarrow$ sQ4.4

Gegeben:

$$B = 5.59375, \quad m = 4$$

Skalieren:

$$5.59375 \cdot 2^4 = 89.5$$

Truncate	Round
$I_{\text{trunc}} = \lfloor 89.5 \rfloor = 89$	$I_{\text{round}} = \lfloor 89.5 + 0.5 \rfloor = 90$
$89_{10} = 01011001_2$	$90_{10} = 01011010_2$
$b_{\text{sfixed}} = 01011001$	$b_{\text{sfixed}} = 01011010$
$89 \cdot 2^{-4} = 5.5625$	$90 \cdot 2^{-4} = 5.625$

4.12 FIR-Filter (Prüfungsrelevanz)

- Multiplikation erzeugt Bitwachstum
- Akkumulation benötigt Guard Bits

4.13 Systematische Wortlängenplanung

- Wertebereich analysieren
- Konsistente Q-Formate
- Guard Bits gezielt einsetzen

Abschluss-Merkatz: Fixed Point ist kontrollierter Fehler. Planung schlägt Wortlänge.

5 Custom IP Blocks: Memory

5.1 Einordnung und Ziel

Speicher sind zentrale Bausteine digitaler Systeme und dominieren häufig Fläche, Energieverbrauch und Performance. Auf FPGAs existieren keine dedizierten ROM-Strukturen. RAM und ROM werden aus vorhandenen Hardware-Ressourcen abgebildet.

Merkatz (Prüfung): RAM und ROM unterscheiden sich in VHDL primär durch das Zugriffsverhalten, nicht durch die Struktur.

Ziel ist die parametrisierbare Beschreibung, Initialisierung und Wiederverwendung von Speicherstrukturen als IP-Blöcke.

5.2 Speichertypen

Grundsätzlich wird zwischen RAM und ROM unterschieden:

- **RAM:** Lesen und Schreiben zur Laufzeit
- **ROM:** Nur Lesen, konstante Inhalte

Technologisch:

- **SRAM:** Statisch, kein Refresh
- **DRAM:** Dynamisch, Refresh erforderlich

FPGA-spezifisch:

- **Distributed RAM** (LUT-basiert)
- **Block RAM** (dedizierte Hard-Macros)

5.3 Speicherorganisation

Logisch wird ein Speicher beschrieben durch:

- **Breite:** Bits pro Wort
- **Tiefe:** Anzahl Wörter

Prüfungsformeln

$$\text{Tiefe} = 2^{\text{ADDR_WIDTH}}$$

$$\text{Gesamtbits} = \text{Breite} \cdot \text{Tiefe}$$

Prüfungsregel: ADDR_ WIDTH bestimmt die Anzahl Adressen, nicht die Wortbreite.

5.4 Speicherports und Zugriffsarten

Unterstützte Port-Typen:

- Single Port RAM
- Simple Dual Port RAM
- True Dual Port RAM

Unterschiede (Prüfung)

- Single Port: Lesen oder Schreiben
- Simple DP: ein Schreib-, ein Leseport
- True DP: zwei vollwertige Ports

5.5 Zugriffsmodi bei Schreibkollisionen

Bei gleichzeitigem Zugriff auf dieselbe Adresse:

- Write First
- Read First
- No Change

Prüfungsregel: Das Verhalten ist nicht implizit, sondern muss spezifiziert werden. RAM Write-Collision Behaviour (Exam)

```
1 process(clk)
2 begin
3   if rising_edge(clk) then
4     if we = '1' then
5       ram(addr) <= din;
6     end if;
7
8     -- WRITE_FIRST
9     dout <= din when we='1' else ram(addr);
10
11    -- READ_FIRST
12    -- dout <= ram(addr);
13
14    -- NO_CHANGE
15    -- if we='0' then dout <= ram(addr); end if;
16  end if;
17 end process;
```

5.6 Distributed RAM

Distributed RAM nutzt LUTs (SLICEMs):

- Schreiben synchron
- Lesen asynchron
- Kleine, schnelle Speicher

Prüfungsregel: A synchron es Lesen Distributed RAM.

5.7 Block RAM

Block RAM besteht aus dedizierten Hard-Macros:

- 36 kb physisch, ca. 32 kb nutzbar
- Synchrones Lesen und Schreiben
- Unterstützt ECC und FIFO

Prüfungsregel: Synchrones Lesen → Block RAM.

Distributed RAM: LUT-basiert, synchrones Schreiben, asynchrones Lesen. Geeignet für kleine, schnelle Speicher. **Prüfungsregel:** async read ⇒ Distributed RAM.

Block RAM: Dedizierte Speicherblöcke, synchrones Lesen und Schreiben. Geeignet für große Speicher mit deterministischem Timing. **Prüfungsregel:** sync read ⇒ Block RAM.

5.8 VHDL-Modellierung von RAM

Grundregeln

- Breite und Tiefe über Generics
- Eindimensionales Array
- Ein Prozess pro Port

Single Port RAM – Code-Schablone

```
19 type ram_t is array (0 to DEPTH-1) of std_logic_vector(W-1 downto
20 0);
21 signal ram : ram_t;
22
23 process(clk)
24 begin
25   if rising_edge(clk) then
26     if we='1' then
27       ram(addr) <= din;
28     end if;
29     dout <= ram(addr);
30   end if;
31 end process;
```

True Dual Port RAM – Hinweis

True Dual Port RAM wird häufig besser über dedizierte Block-RAM-IP realisiert. Eine VHDL-Modellierung mit shared variable ist simulatorfreundlich, aber kritisch.

5.9 Synthesis-Attribute

Die Implementierung kann gezielt beeinflusst werden:

```
31 attribute ram_style : string;
32 attribute ram_style of ram : signal is "block";
```

Alternativ:

```
33 attribute ram_style of ram : signal is "distributed";
```

Merksatz (Prüfung): Attribute sind Hinweise an das Tool, keine funktionalen Anweisungen.

5.10 ROM-Implementierung

ROM unterscheidet sich von RAM nur durch konstante Inhalte.

ROM – Code-Schablone

```
34 type rom_t is array (0 to 15) of std_logic_vector(7 downto 0);
35 constant rom : rom_t := ( -- manuelle Initialisierung
36   0 => x"12",
37   1 => x"34",
38   others => x"00"
39 );
40 -- oder dateibasierte Initialisierung
41 impure function init_mem(mif_file_name : string) return rom_t is
42   file mif_file : text open read_mode is mif_file_name;
43   variable mif_line : line;
44   variable temp_bv : bit_vector (7 downto 0);
45   variable temp_mem : rom_t;
46 begin
47   for i in 0 to rom_t'range loop
48     readline(mif_file, mif_line);
49     read(mif_line, temp_bv);
50     temp_mem(i) := std_logic_vector(temp_bv);
51   end loop;
52   return temp_mem;
53 end function;
54 constant rom : rom_t := init_mem("my_file.mif");
```

5.11 Speicherinitialisierung

Möglichkeiten:

- Initialisierung bei Deklaration
- Datei-basierte Initialisierung (.coe, .mif)

Prüfungsregel: ROM ohne Initialisierung ist inhaltlich wertlos.

5.12 Memory Generator IP

Parametrisierte Speicher-IP mit wählbarer Breite, Tiefe und Zugriffsart wird von Xilinx bereitgestellt.

5.13 IP-Wiederverwendung und Integration

IP-Blöcke sind parametrisierte, wiederverwendbare Einheiten.

- Hard IP
- Firm IP
- Soft IP

Integration:

- IP Integrator
- RTL-Instantiation

IP-Integrator / ARM-Workflow (Zynq): Das Processing System (ARM) wird im IP Integrator konfiguriert und über AXI mit Custom-IP in der Programmable Logic verbunden. Nach Validierung des Block Designs erfolgt die Hardware-Generierung und der Export (XSA/HDF) für die Software-Toolchain. Software (BSP, Treiber, Applikation) greift über memory-mapped Register auf die FPGA-Logik zu.

IP Life Cycle (Prüfung) Design → Verifikation → Packaging → Integration → Reuse

6 Serial Communication Interfaces

6.1 Serielle Schnittstellen – kompakte Einordnung

Zentrale Regel: Serielle Kommunikation spart Pins, erhöht Logikaufwand.

Grundklassifikation (Prüfung):

- **Topologie:** Point-to-Point, Bus
- **Zeitorganisation:** synchron, asynchron
- **Physik:** Single-Ended, differentiell

Prüfungsregel: Serielle Schnittstellen reduzieren Pin-Anzahl, benötigen jedoch Serialisierung und Steuerlogik.

Logik zu Physik:

- VHDL beschreibt **logische Signale**
- Elektrisches Verhalten wird durch **I/O-Pads und Constraints** festgelegt

Prüfungsregel: Das Pad bestimmt Spannungspegel, Drive Strength und Pull-Ups, **nicht das Protokoll**.

Spannungs- vs. Strommodus (Kurzfassung):

- **Voltage Mode:** Spannungspegel, einfach, geringere

Datenrate

- **Current Mode:** Stromfluss, kleiner Signalhub, hohe Datenrate

Prüfungsregel: Hohe Datenrate ⇒ **Current Mode** (z.B. LVDS).

LVDS (nur prüfungsrelevant):

- Differenziell
- Strommodus
- Hohe Störfestigkeit

Prüfungsmerksatz: LVDS wird über **I/O-Constraints** konfiguriert, nicht im HDL.

Serialisierung (Minimalmodell):

- Parallel-In / Serial-Out (PISO)
- Serial-In / Parallel-Out (SIPO)
- Implementiert mit **Shiftregistern + FSM**

Prüfungsregel: Serielle Übertragung benötigt explizite **Send-/Empfangssteuerung**.

6.2 SPI – Serial Peripheral Interface

SPI ist eine synchrone, serielle Punkt-zu-Punkt-Schnittstelle.

Signale:

- CS, SCLK, MOSI, MISO

Eigenschaften:

- Master-Slave
- Vollduplex

Übertragungsparameter:

- Clock Polarity (CPOL)
- Clock Phase (CPHA)

6.3 SPI-Hardwarestruktur

SPI besteht aus Shiftregistern, Clock-Divider und FSM.

Prüfungsregel: SPI ist **vollständig synchron**.

6.4 SPI-Beschreibung in VHDL

Die Beschreibung folgt einem festen Muster: Generics für Wortbreite und Modus, FSM zur Steuerung und Clock-Divider für SCLK.

6.5 I²C – Inter-Integrated Circuit

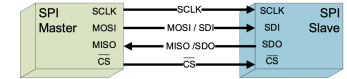
I²C ist eine synchrone Zweidraht-Bus-Schnittstelle.

Eigenschaften:

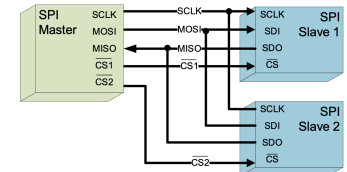
- SDA, SCL
- Multi-Master, Multi-Slave
- Open-Drain

Serial Peripheral Interface

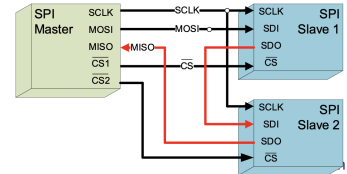
- **Single Point – to – Point**



- **Multi – subnode SPI configuration (star with individual CS – lines)**



- **Daisy-Chain SPI configuration (one shared CS – line)**



Digital Microelectronics, HS 2025

Abbildung 2: SPI-Topologien, Signale und Master-Slave-Prinzip.

6.6 I²C (Prüfungsrelevanz)

- SDA, SCL (Open-Drain)
- Start/Stop bei SCL=High
- Arbitration über Wired-AND

Open-Drain in VHDL:

```
55 sda <= '0' when drive_low='1' else 'Z';
```

6.7 UART (Asynchrone serielle Schnittstelle)

UART ist eine **asynchrone** Punkt-zu-Punkt Schnittstelle ohne gemeinsame Clock. Die Übertragung erfolgt rahmenbasiert: Startbit (0), 8 Datenbits (LSB first), optional Parity, Stopbit (1). Typisch: 8N1.

Timing / Baudrate Die Bitzeit beträgt $T_b = 1/\text{baud}$. In synchroner Logik wird ein Baud-Tick über einen Clock-

Teiler erzeugt:

$$\text{BAUD_DIV} = \left\lfloor \frac{f_{\text{clk}}}{\text{baud}} \right\rfloor$$

Sender (TX) TX wird mit einer FSM und einem Shiftregister realisiert: IDLE (Leitung high) → START → DATA (8 Bits) → STOP. Pro Baud-Tick wird ein Bit ausgegeben.

Empfänger (RX) RX erkennt das Startbit (fallende Flanke) und sampelt jedes Bit in der Bitmitte (optional mit Oversampling). Nach korrektem Stopbit wird rx_valid gesetzt.

Prüfungsregel RX-Sampling muss in der Bitmitte erfolgen; andernfalls steigt die Bitfehlerrate deutlich an.

Prüfungsmerksätze UART ist asynchron. Timing entsteht aus dem Baud-Teiler. Daten werden LSB first übertragen. Implementierung erfolgt mit FSM + Shiftregister.

7 Parallel On-Chip Communication

7.1 Einordnung paralleler Kommunikation

Parallele Kommunikation überträgt mehrere Bits gleichzeitig über separate Leitungen. Sie wird primär **on-chip** eingesetzt, da physikalische Effekte off-chip die Skalierbarkeit stark begrenzen.

Merksatz (Prüfung): Parallel = hoher Durchsatz bei kurzer Distanz.

7.2 Serial vs. Parallel

Prüfungsregel: Off-chip dominiert seriell, on-chip dominiert parallel.

7.3 Physikalische Grenzen paralleler Off-Chip-Kommunikation

Außerhalb des Chips begrenzen physikalische Effekte den Nutzen paralleler Übertragung:

- **Skew:** Laufzeitunterschiede zwischen Leitungen
- **Crosstalk:** Kopplung benachbarter Leitungen
- Hoher Pad-Bedarf

- Hohe Kosten

Prüfungsregel: Mehr Leitungen → mehr Off-Chip-Probleme.

7.4 Vorteile paralleler On-Chip-Kommunikation

Auf dem Chip entfallen die meisten Nachteile: Keine Pads, kurze kontrollierte Leitungen, geringer Skew und keine SERDES-Strukturen.

Merksatz: On-Chip-Interconnects sind natürlich parallel.

7.5 Grundstruktur paralleler Schnittstellen

Eine parallele Schnittstelle besteht aus:

- Datenbus
- Steuerleitungen (Read, Write, Valid, Ready)
- Gemeinsamer Clock

Prüfungsregel: Parallele Interfaces sind immer synchron.

7.6 AXI als Standard-On-Chip-Interconnect

AXI (Advanced eXtensible Interface) ist Teil der AMBA-Architektur von ARM und de-facto Standard für SoC-Designs.

- Master-Slave
- Memory-Mapped
- Hohe Parallelität

Merksatz (Prüfung): AXI ist kanalbasiert, kein klassischer Bus.

7.7 AXI-Architektur

AXI verwendet fünf unabhängige Kanäle: Write Address (AW), Write Data (W), Write Response (B), Read Address (AR) und Read Data (R).

Prüfungsregel: Read und Write sind vollständig getrennt.

7.8 AXI Handshake (VALID/READY)

Zentrale Regel: Alle AXI-Kanäle nutzen dasselbe Handshake-Schema.

1. **Source** setzt VALID=1, sobald Adresse/Daten stabil anliegen.
2. **Destination** setzt READY=1, sobald sie Adresse/Daten annehmen kann.
3. **Transfer** erfolgt an der ersten aktiven Taktflanke, nachdem VALID=1 und READY=1 gleichzeitig sind.

Prüfungsregel: Solange VALID=1 und READY=0, müssen Adresse/Daten stabil bleiben.

7.9 AXI-Varianten

AXI existiert in mehreren Varianten:

- **AXI4:** Vollständig, Bursts bis 256 Transfers
- **AXI4-Lite:** Keine Bursts, Registerzugriffe
- **AXI4-Stream:** Datenstrom ohne Adressen

Prüfungsregel: Register-IP → AXI4-Lite.

7.10 AXI-Interconnect-Topologien

AXI-Interconnects verbinden mehrere Master und Slaves über definierte Topologien. Je nach Richtung sind unterschiedliche Kontrollmechanismen notwendig.

1-to-1: Ein Master direkt mit einem Slave verbunden (ohne Arbitration oder Adressdekodierung).

N-to-1 (Arbitration): Mehrere Master greifen auf einen Slave zu (z.B. Speichercontroller, nur ein Master darf gleichzeitig senden).

1-to-M (Decoding/Routing): Ein Master greift auf mehrere Slaves zu (z.B. CPU auf verschiedene Peripheriekomponenten, Adressdekodierung, keine Arbitration).

N-to-M: Mehrere Master greifen auf mehrere Slaves zu (Arbitration und Adressdekodierung).

Prüfungsregel: AXI-Interconnects sind punkt-zu-punkt kanalbasiert; Arbitration und Routing erfolgen im Interconnect, nicht im Master.

7.11 AXI auf Zynq

PS und PL sind über General Purpose (GP), High Performance (HP) und Accelerator Coherency Port (ACP) AXI-Ports verbunden.

7.12 AXI3 vs. AXI4

Im Zynq:

- PS nutzt AXI3
- Xilinx-IP nutzt AXI4

Prüfungsregel: Protokollkonverter lösen Inkompatibilitäten .

7.13 AXI-IP-Erstellung

Gängige Vorgehensweisen:

- Separater AXI-Interface-Block + User-Logik
- AXI-Peripheral-Template

7.14 Register-basierte Kommunikation

Kommunikation erfolgt über Memory-Mapped Register:

- CPU schreibt Register
- Logik liest Register
- Logik schreibt Status
- CPU liest Status

7.15 FPGA-Processor-Kommunikation

Integration erfolgt über:

- Hardware-Export
- BSP-Generierung
- Automatische Address-Header

Abschluss-Merkatz: AXI verbindet Software-Welt und FPGA-Logik deterministisch.

8 Debugging and Verification of SoC

8.1 Einordnung der Verifikation

Design Verification ist ein zentraler Bestandteil moderner SoC-Entwicklung und beansprucht häufig über 50% der Entwicklungszeit.

Prüfungsmerksatz: Verifikation prüft richtiges Verhalten, nicht Performance oder Timing .

In dieser Vorlesung wird ausschließlich funktionale Verifikation betrachtet.

8.2 Funktionale Verifikation

Funktionales Verhalten lässt sich beschreiben als:

$$y = f(x, \text{state})$$

Prüfungsregel: Verifikation prüft , ob für gegebene Eingänge und Zustände die korrekten Ausgänge entstehen.

8.3 Exhaustive Verification

Exhaustive Verification testet alle möglichen Eingangskombinationen.

Prüfungsformel

Für n unabhängige Eingänge gilt:

$$N = 2^n$$

Erweitert (Prüfung):

$$N = 2^{(n_{inputs} + n_{state})}$$

Beispiel: 1 Input + 2 Zustände $\rightarrow 2^3 = 8$ Testvektoren

Prüfungsregel: Exhaustive Verification ist theoretisch korrekt, praktisch unmöglich.

8.4 Pragmatische Verifikationsstrategie

Da Exhaustive Verification nicht realisierbar ist, wird ein Kompromiss gewählt:

- Assertion-Based Verification
- Gezielte Testfälle
- Zufällige Stimuli
- Trennung von Design und Test

Merksatz: Ziel ist hohe Fehlerabdeckung , nicht Vollständigkeit.

8.5 Assertion-Based Verification

Assertions sind eingebaute Konsistenzprüfungen.

Typische Assertions

- Illegale FSM-Zustände
- Ungültige Adressen
- Über- und Unterläufe

Assertion – Code-Beispiel

```
56 assert addr < MAX_ADDR
57 report "Address out of range"
58 severity error;
```

Prüfungsregel: Assertions sind nicht für die Synthese bestimmt .

8.6 Testfälle und Stimuli

Effektive Testfälle decken unterschiedliche Szenarien ab:

- Normalbetrieb
- Grenzfälle
- Fehlerfälle
- Zufällige Daten

Prüfungsregel: Grenzwerte finden mehr Fehler als Mittelwerte.

8.7 Golden Model

Ein Golden Model liefert Referenzergebnisse für gegebene Stimuli.

Prüfungsregel

DUT erzeugt Ergebnis $\rightarrow$ Golden Model erzeugt Erwartungswert $\rightarrow$ Automatischer Vergleich.

Ohne Golden Model keine skalierbare Verifikation.

8.8 Test-Bench-Organisation

Strukturierte Test-Benches folgen festen Regeln:

- Periodische Clock
- Genau ein Stimulus pro Takt
- Antwortauswertung synchron zur Clock

Clock – Minimal-Code

```
59 clk <= not clk after 10 ns;
```

Prüfungsregel: Un synchron isierte Test-Benches erzeugen Scheinergebnisse. **Testbench – Exam Skeleton**

```
1 signal clk : std_logic := '0';
2
3 clk <= not clk after 10 ns;
4
5 stim : process
```



```

6 begin
7   rst <= '1'; wait for 20 ns;
8   rst <= '0';
9   din <= x"55"; wait for 20 ns;
10  assert dout = x"55"
11    report "Mismatch"
12    severity error;
13  wait;
14 end process;

```

8.9 Automatische Auswertung

Visuelle Inspektion ist unzureichend.

Prüfungsrezept

Stimulus erzeugen → DUT simulieren → Ergebnis speichern
→ mit Golden Model vergleichen.

Merksatz: Was nicht automatisch prüft, skaliert nicht.

8.10 File-basierte Test-Benches

Erlauben große Testmengen. Nur in Testbenches erlaubt.

Prüfungsregel: File-I/O ist Testbench-only.

8.11 Simulation und Modelle

Funktionale Simulation:

- VHDL oder Verilog

Timing-Simulation:

- Nur Verilog

Prüfungsregel: VHDL besitzt keine simprim-Timing bibliothek.

8.12 Debugging in Simulation

Simulation erlaubt vollständige Sichtbarkeit. Mit den Werkzeugen Signal-Inspection, Breakpoints, Step-by-Step und Signal-Forcing können Fehler systematisch gefunden werden.

8.13 Voraussetzungen zum Finden eines Fehlers

Ein Fehler wird nur gefunden, wenn alle drei Bedingungen erfüllt sind:

1. **Sensitization:** Fehler wird aktiviert
2. **Propagation:** Fehler breitet sich aus
3. **Observation:** Fehler ist sichtbar

Prüfungsmerksatz

Kein Output-Unterschied ⇒ kein gefundener Fehler.

8.14 Stuck-at-Fehlermodell

Ein Signal ist permanent auf 0 oder 1 festgesetzt.

Prüfungsrezept

- Eingang so wählen, dass sa0/sa1 wirksam wird
- Pfad bis Ausgang freischalten
- Ausgang vergleichen

Prüfungsregel: Stuck-at ist ein logisches, kein physikalisches Modell. Kein Output-Unterschied ⇒ kein gefundener Fehler.

8.15 Debugging in Emulation

Auf Hardware sind interne Signale nicht direkt sichtbar.

Xilinx-Werkzeuge

- VIO: Setzen und Lesen interner Signale
- ILA: Logikanalysator im FPGA

Prüfungsregel: ILA ersetzt den Simulator auf Hardware.

8.16 Hardware/Software Co-Debugging

In SoCs ist gekoppeltes Debugging möglich.

Mechanismus

- Software-Breakpoint triggert ILA
- ILA-Trigger stoppt CPU

Abschluss-Merksatz: Gute Verifikation findet Fehler früh, billig und reproduzierbar.

9 Exam Code Templates

9.1 XDC: Physical + Timing (Skeleton)

```

1 ## Physical constraints
2 set_property IOSTANDARD LVCMOS33 [get_ports {clk rst}]
3 set_property PACKAGE_PIN W17 [get_ports clk]
4 set_property PACKAGE_PIN W20 [get_ports rst]

```

```

5 set_property PULLUP true [get_ports rst]
6
7 ## Clock (waveform)
8 create_clock -name clk -period 10.000 -waveform {7.000
9   10.000} [get_ports clk]
10
11 ## Input delays ( relative to clk, bundle x[*])
12 set_input_delay -clock [get_clocks clk] -min -add_delay
13   1.000 [get_ports {x[*]}]
14 set_input_delay -clock [get_clocks clk] -max -add_delay
15   2.500 [get_ports {x[*]}]
16
17 ## Output delays (keep as generic placeholders)
18 # set_output_delay -clock [get_clocks clk] -min 0.5
19   [get_ports {y[*]}]
20 # set_output_delay -clock [get_clocks clk] -max 2.0
21   [get_ports {y[*]}]

```

9.2 VHDL: Simple Dual Port RAM (Block RAM inference)

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.numeric_std.all;
4
5 entity sdp_ram is
6   generic(
7     ADDR_WIDTH : positive := 16;
8     DATA_WIDTH : positive := 8
9   );
10  port(
11    clk_wr : in std_ulogic;
12    clk_rd : in std_ulogic;
13    we : in std_ulogic;
14    addr_wr : in std_ulogic_vector(ADDR_WIDTH-1 downto 0);
15    addr_rd : in std_ulogic_vector(ADDR_WIDTH-1 downto 0);
16    din : in std_ulogic_vector(DATA_WIDTH-1 downto 0);
17    dout : out std_ulogic_vector(DATA_WIDTH-1 downto 0)
18  );
19 end entity;
20
21 architecture rtl of sdp_ram is
22   type mem_t is array (0 to 2**ADDR_WIDTH - 1) of
23     std_ulogic_vector(DATA_WIDTH-1 downto 0);
24   shared variable mem : mem_t;
25
26   attribute ram_style : string;
27   attribute ram_style of mem : variable is "block";
28 begin
29   -- Write port (sync)
30   p_wr : process(clk_wr)
31   begin
32     if rising_edge(clk_wr) then
33       if we = '1' then
34         mem(to_integer(unsigned(addr_wr))) := din;
35       end if;
36     end if;
37   end process;
38
39   -- Read port (sync)
40   p_rd : process(clk_rd)
41   begin
42     if rising_edge(clk_rd) then

```

```

42     dout <= mem(to_integer(unsigned(addr_rd)));
43   end if;
44   end process;
45 end architecture;

```

9.3 VHDL: I2C Open-Drain (one wire)

```

1  entity i2c_line is
2    port(
3      sda      : inout std_logic;
4      drive_lo : in    std_logic; -- '1' => pull low
5      sda_in   : out   std_logic
6    );
7  end entity;
8
9  architecture rtl of i2c_line is
10   begin
11     -- Open-drain: never drive '1'
12     sda <= '0' when drive_lo = '1' else 'Z';
13     sda_in <= sda;
14   end architecture;

```

9.4 VHDL: AXI4-Lite Registerbank (minimal)

```

1  -- AXI4-Lite Registerbank (Exam-Minimal, 1-cycle ready, no stalls)
2  -- Idea: regs[] memory-mapped, addr word-aligned: addr(5 downto 2)
3
4  type regfile_t is array (0 to 7) of std_ulogic_vector(31 downto 0);
5  signal regs : regfile_t := (others => (others => '0'));
6  signal widx : integer range 0 to 7;
7  signal ridx : integer range 0 to 7;
8
9  widx <= to_integer(unsigned(awaddr(5 downto 2)));
10 ridx <= to_integer(unsigned(araddr(5 downto 2)));
11
12 awready <= '1'; wready <= '1'; arready <= '1'; -- always ready
13   (no backpressure)
14
15 process(aclk)
16   begin
17     if rising_edge(aclk) then
18       if aresetn='0' then
19         regs <= (others => (others => '0'));
20         bvalid <= '0';
21         rvalid <= '0';
22         rdata <= (others => '0');
23       else
24         -- clear responses when accepted
25         if bvalid='1' and bready='1' then bvalid <= '0'; end if;
26         if rvalid='1' and rready='1' then rvalid <= '0'; end if;
27
28         -- WRITE: accept when AW+W valid in same cycle (minimal)
29         if (awvalid='1') and (wvalid='1') then
30           regs(widx) <= wdata;
31           bvalid <= '1';
32         end if;
33
34         -- READ: respond immediately on AR valid (minimal)
35         if (arvalid='1') then
36           rdata <= regs(ridx);
37           rvalid <= '1';
38         end if;
39     end if;

```

```

40 end process;

```

UART TX – Minimal Code Skeleton

```

1  -- UART TX: Baud + FSM in einem Prozess (Exam-Minimal)
2  -- generics: CLK_HZ, BAUD
3
4  type state_t is (IDLE, START, DATA, STOP);
5  signal state : state_t := IDLE;
6
7  constant BAUD_DIV : integer := CLK_HZ / BAUD;
8  signal cnt : integer range 0 to BAUD_DIV-1 := 0;
9  signal bit_idx : integer range 0 to 7 := 0;
10 signal shreg : std_logic_vector(7 downto 0);
11
12 process(clk)
13   begin
14     if rising_edge(clk) then
15       -- Baud tick
16       if cnt = BAUD_DIV-1 then
17         cnt <= 0;
18
19       case state is
20         when IDLE =>
21           tx <= '1';
22           if tx_start='1' then
23             shreg <= tx_data;
24             bit_idx <= 0;
25             state <= START;
26           end if;
27
28         when START =>
29           tx <= '0';
30           state <= DATA;
31
32         when DATA =>
33           tx <= shreg(0);
34           shreg <= '0' & shreg(7 downto 1);
35           if bit_idx=7 then
36             state <= STOP;
37           else
38             bit_idx <= bit_idx + 1;
39           end if;
40
41         when STOP =>
42           tx <= '1';
43           state <= IDLE;
44       end case;
45
46     else
47       cnt <= cnt + 1;
48     end if;
49   end if;
50 end process;

```

Merksatz (Prüfung): UART ist asynchron. Timing entsteht aus dem Baud-Teiler. Implementierung: FSM + Shift register.

9.5 Exam Code Patterns (Fixed Point & Package)

```

1  -- Package: zentrale Definitionen
2  package exam_pkg is
3    constant W : integer := 8;

```

```

4    function add_fp(a,b : signed(W-1 downto 0)) return signed;
5  end package;
6
7  package body exam_pkg is
8    function add_fp(a,b : signed(W-1 downto 0)) return signed is
9      variable ext : signed(W downto 0); -- Guard Bit
10   begin
11     ext := (a(a'left) & a) + (b(b'left) & b);
12     return ext(W-1 downto 0); -- Trunkierung
13   end function;
14 end package body;
15
16 -- Nutzung im synchronen Prozess
17 use work.exam_pkg.all;
18
19 process(clk)
20   begin
21     if rising_edge(clk) then
22       y <= add_fp(a,b);
23     end if;
24   end process;

```

Schieberegister (Shift Register)

```

1  process(clk)
2   begin
3     if rising_edge(clk) then
4       if rst='1' then
5         reg <= (others => '0');
6       elsif en='1' then
7         reg <= reg(reg'left-1 downto 0) & din;
8       end if;
9     end if;
10   end process;

```

Register mit Enable (Latch-sicher)

```

1  process(clk)
2   begin
3     if rising_edge(clk) then
4       q <= q; -- default
5       if en='1' then
6         q <= d;
7       end if;
8     end if;
9   end process;

```

Reset-Varianten (Prüfung)

```

1  -- synchronous reset
2  if rising_edge(clk) then
3    if rst='1' then
4      q <= '0';
5    else
6      q <= d;
7    end if;
8  end if;
9
10 -- asynchronous reset
11 if rst='1' then
12   q <= '0';
13 elsif rising_edge(clk) then
14   q <= d;
15 end if;

```

Clock Enable vs. Gated Clock Clock Enable bevorzugen. Gated Clock vermeidet man wegen Glitches.